

CSED 312 Project 3 Design Report

김지현(20220302)

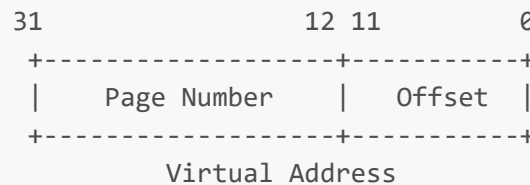
전현빈(20220259)

Analyze the Current Implementation

이번 프로젝트 3에서는 virtual memory를 다루며, 이를 이해하기 위해 memory와 storage와 관련된 네 가지 주요 개념에 대한 이해가 필요하다. Pintos 문서에서는 이 네 가지 개념을 우선적으로 설명하고 있으므로, 본 디자인 보고서에서도 해당 개념들을 먼저 다룬 후, 각 알고리즘과 구조체를 설명할 것이다.

1. Page

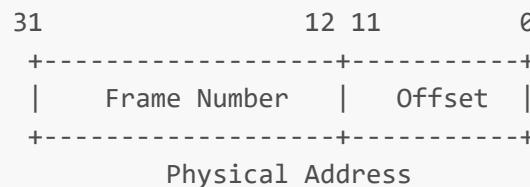
Page는 virtual page를 의미하며, 4,096($=2^{12}$)바이트 크기의 연속적인 가상 메모리 영역이다. 32비트 가상 주소는 페이지 번호(page number)와 오프셋(offset) 두 부분으로 나뉜다. 따라서 한 페이지 내의 임의의 주소는 12bit의 offset주소로 가르킬 수 있다. 따라서 32bit Virtual Address(VA라 함)은 아래와 같은 구조를 갖는다.



각 프로세스는 서로 다른 user virtual page를 가지며, kernel page는 모든 프로세스에서 동일하다. kernel process는 사용자 페이지와 커널 페이지 모두에 접근할 수 있는 반면, user process는 자신의 사용자 페이지에만 접근할 수 있다.

2. Frames

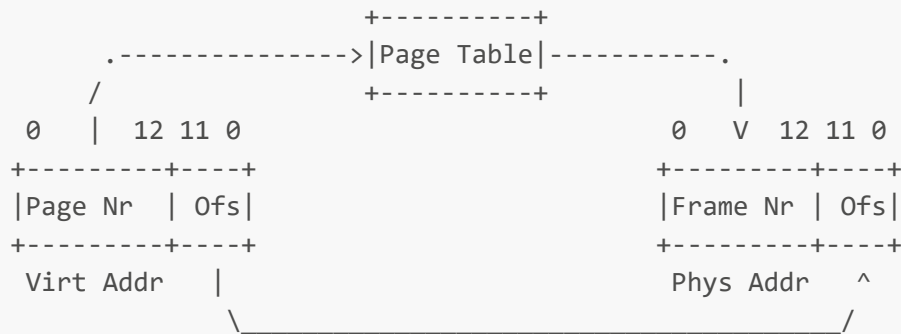
Frame은 physical frame 또는 page frame을 의미하며, physical memory의 연속적인 영역을 나타낸다. Page와 유사하게, 32비트 physical address(PA라 함)는 프레임 번호(frame number)와 오프셋(offset)으로 나뉜다.



이때 frame size와 page size가 같으므로 Pintos에서는 page를 frame과 mapping하여 해당 memory에 접근할 수 있도록 구현되어 있다.

3. Page Tables

Page table은 앞서 설명한 page와 frame 간의 mapping 과정에서 사용되는 data structure이다. 이후 설명할 `pagedir.c` 코드에서 상세히 다루겠지만, 간단히 설명하자면, page table은 page number를 frame number로 mapping하며, offset은 page와 frame에서 동일한 값을 갖도록 구현되어 있다. 이를 통해 VA를 PA로 translate 할 수 있다.



4. Swap Slots

Swap slot은 disk의 swap partition에서 page size만큼의 연속적인 영역을 의미한다.

위와 같이, 프로젝트 3에서 다루는 virtual memory의 핵심 개념인 Page, Frame, Page Table, 그리고 Swap Slot에 대해 설명하였다. 아래에서는 이 개념들을 기반으로 한 알고리즘 및 구조체 구현 방식을 다룰 것이다.

먼저, page를 할당할 때 사용되는 주요 함수들에 대해 알아본다. 앞서 설명한 page와 frame은 모두 page-size 단위로 메모리가 할당되며, 이 과정에서 palloc 함수가 사용된다. Pintos의 시작점인 `init.c` 코드의 `main` 함수를 살펴보면, `palloc_init` 함수를 통해 전체 free memory를 계산한 후, 이를 user memory와 kernel memory로 나누는 것을 확인할 수 있다. 이후, 각각의 메모리는 `init_pool` 함수를 통해 초기화된다. 이 초기화 과정에서 사용자와 커널 메모리는 별도의 memory pool로 관리된다.

```
main (void)
{
    /* Initialize memory system. */
    palloc_init (user_page_limit);
    ...
}

void
palloc_init (size_t user_page_limit)
{
    /* Free memory starts at 1 MB and runs to the end of RAM. */
    uint8_t *free_start = ptov (1024 * 1024);
    uint8_t *free_end = ptov (init_ram_pages * PGSIZE);
    size_t free_pages = (free_end - free_start) / PGSIZE;
    size_t user_pages = free_pages / 2;
    size_t kernel_pages;
    if (user_pages > user_page_limit)
        user_pages = user_page_limit;
    kernel_pages = free_pages - user_pages;

    /* Give half of memory to kernel, half to user. */
    init_pool (&kernel_pool, free_start, kernel_pages, "kernel pool");
    init_pool (&user_pool, free_start + kernel_pages * PGSIZE,
              user_pages, "user pool");
}
```

위에서 설명한 사용자와 커널 메모리를 초기화하기 위해 사용되는 `init_pool` 함수는 각 페이지의 사용 여부를 관리하는 bitmap을 설정한다. 이 함수에서는 전체 페이지 중 일부를 비트맵 저장 공간으로 사용하며, `bitmap_create_in_buf` 함수를 통해 비트맵을 초기화한다. 이후, 메모리 할당이 시작되는 주소를 `base + bm_pages * PGSIZE`로 설정하여 비트맵 영역과 할당 영역을 구분한다. 또한, 멀티스레드 환경에서 race condition을 방지하기 위해 메모리 풀 구조체의 락(`p->lock`)을 초기화한다.

```
static void
init_pool (struct pool *p, void *base, size_t page_cnt, const char *name)
{
    /* We'll put the pool's used_map at its base.
       Calculate the space needed for the bitmap
       and subtract it from the pool's size. */
    size_t bm_pages = DIV_ROUND_UP (bitmap_buf_size (page_cnt), PGSIZE);
    if (bm_pages > page_cnt)
        PANIC ("Not enough memory in %s for bitmap.", name);
    page_cnt -= bm_pages;

    printf ("%zu pages available in %s.\n", page_cnt, name);

    /* Initialize the pool. */
    lock_init (&p->lock);
    p->used_map = bitmap_create_in_buf (page_cnt, base, bm_pages * PGSIZE);
    p->base = base + bm_pages * PGSIZE;
}
```

다음으로 `pallocc.c`의 다른 함수들을 살펴보았다. 먼저, `pallocc_get_page` 함수는 하나의 페이지를 할당하는 함수로, 내부적으로 `pallocc_get_multiple()` 함수를 호출한다. 이때, 1을 인자로 넘겨 하나의 페이지만 할당 되도록 한다.

`pallocc_get_multiple` 함수에서는 먼저 인자로 전달받은 `flag` 값이 `PAL_USER`인지 확인한다. `PAL_USER`인 경우 사용자 메모리 풀에서 메모리를 할당하도록 `pool` 포인터를 설정하고, 그렇지 않으면 커널 메모리 풀에서 메모리를 할당하도록 설정한다. 이후, `lock`을 사용해 atomic 하게 `bitmap_scan_and_flip` 함수를 호출하여, `page_cnt`만큼 연속적으로 비어 있는 페이지를 검색하고 이를 `used` 상태로 설정한다.

할당 가능한 페이지가 존재할 경우, 해당 페이지의 첫 번째 인덱스를 반환하며, 페이지가 존재하지 않으면 `BITMAP_ERROR`를 반환하여 에러를 처리한다. `lock`을 해제한 후, 반환받은 인덱스를 이용해 실제 할당된 메모리의 시작 주소를 계산한다. 이 계산은 현재 메모리 풀의 시작 주소에 (`페이지 크기 * 인덱스`)를 더하는 방식으로 이루어진다.

마지막으로, `pages`가 정상적으로 할당된 후 `flags` 값을 확인하여, 메모리를 0으로 초기화해야 할 경우 `memset` 함수를 사용해 초기화한다.

```
void *
pallocc_get_page (enum pallocc_flags flags)
{
    return pallocc_get_multiple (flags, 1);
}

void *
```

```

palloc_get_multiple (enum palloc_flags flags, size_t page_cnt)
{
    struct pool *pool = flags & PAL_USER ? &user_pool : &kernel_pool;
    void *pages;
    size_t page_idx;

    if (page_cnt == 0)
        return NULL;

    lock_acquire (&pool->lock);
    page_idx = bitmap_scan_and_flip (pool->used_map, 0, page_cnt, false);
    lock_release (&pool->lock);

    if (page_idx != BITMAP_ERROR)
        pages = pool->base + PGSIZE * page_idx;
    else
        pages = NULL;

    if (pages != NULL)
    {
        if (flags & PAL_ZERO)
            memset (pages, 0, PGSIZE * page_cnt);
    }
    else
    {
        if (flags & PAL_ASSERT)
            PANIC ("palloc_get: out of pages");
    }

    return pages;
}

```

`palloc_free_page`는 하나의 페이지 메모리를 release하는 함수이다. 위와 유사하게 `palloc_free_multiple` 함수를 호출하여 여러 페이지를 해제할 수 있다. `palloc_free_multiple` 함수에서는 먼저 `page_from_pool` 함수를 사용해 해당 페이지(`page`)가 `kernel`에 속하는지, 혹은 `user`에 속하는지 확인한 뒤, 적절한 `pool` 포인터에 저장한다. 이후, 메모리 주소를 `pg_no` 함수를 통해 페이지 단위로 변환하여 페이지 번호를 계산한다. 이 계산 과정은, 현재 페이지의 메모리 주소에서 메모리 풀(pool)의 시작 주소를 빼주는 방식으로 이루어진다. 마지막으로, `bitmap_all` 함수를 통해 해당 페이지가 현재 `used` 상태인지 확인한 뒤, `bitmap_set_multiple` 함수를 사용하여 해제하려는 페이지를 `unused` 상태로 변경한다.

```

void
palloc_free_page (void *page)
{
    palloc_free_multiple (page, 1);
}

void
palloc_free_multiple (void *pages, size_t page_cnt)
{
    struct pool *pool;
    size_t page_idx;

```

```

ASSERT (pg_ofs (pages) == 0);
if (pages == NULL || page_cnt == 0)
    return;

if (page_from_pool (&kernel_pool, pages))
    pool = &kernel_pool;
else if (page_from_pool (&user_pool, pages))
    pool = &user_pool;
else
    NOT_REACHED ();

page_idx = pg_no (pages) - pg_no (pool->base);

#ifdef NDEBUB
    memset (pages, 0xcc, PGSIZE * page_cnt);
#endif

ASSERT (bitmap_all (pool->used_map, page_idx, page_cnt));
bitmap_set_multiple (pool->used_map, page_idx, page_cnt, false);
}

```

`page_from_pool` 함수는 앞서 설명한 `pallocc_free_multiple` 함수에서 사용되며, 전달받은 페이지가 `user memory pool`에 속하는지 또는 `kernel memory pool`에 속하는지를 판별하여 반환하는 함수이다.

이 함수는 인수로 전달받은 `page`의 주소가 해당 메모리 풀의 시작 페이지 주소보다 크고, 마지막 페이지 주소보다 작음을 확인한 뒤, 결과를 `boolean` 값으로 반환한다.

```

static bool
page_from_pool (const struct pool *pool, void *page)
{
    size_t page_no = pg_no (page);
    size_t start_page = pg_no (pool->base);
    size_t end_page = start_page + bitmap_size (pool->used_map);

    return page_no >= start_page && page_no < end_page;
}

```

`load_segment` 함수는 프로세스의 메모리 공간에 파일 데이터를 저장하는 역할을 한다. 먼저, 프로젝트 2에서 사용했던 `file_seek` 함수를 이용해 파일 읽기의 시작 위치를 `ofs`로 설정한다.

이후, `read_bytes` 또는 `zero_bytes`가 남아 있는 동안 `while` 루프를 실행한다. 루프 안에서는 `file_read` 함수를 호출해 `page_read_bytes`만큼 데이터를 읽고, 이를 `kpage`에 저장한다.

데이터를 성공적으로 `page_read_bytes`만큼 읽었다면, 현재 페이지에서 남은 바이트를 0으로 초기화한다. 그러나 만약 `page_read_bytes`만큼 데이터를 읽지 못한 경우, `pallocc_free_page`를 호출해 할당된 페이지를 해제한다.

```

static bool
load_segment (struct file *file, off_t ofs, uint8_t *upage,
              uint32_t read_bytes, uint32_t zero_bytes, bool writable)
{
    ASSERT ((read_bytes + zero_bytes) % PGSIZE == 0);
    ASSERT (pg_ofs (upage) == 0);
    ASSERT (ofs % PGSIZE == 0);

    file_seek (file, ofs);
    while (read_bytes > 0 || zero_bytes > 0)
    {
        /* Calculate how to fill this page.
           We will read PAGE_READ_BYTES bytes from FILE
           and zero the final PAGE_ZERO_BYTES bytes. */
        size_t page_read_bytes = read_bytes < PGSIZE ? read_bytes : PGSIZE;
        size_t page_zero_bytes = PGSIZE - page_read_bytes;

        /* Get a page of memory. */
        uint8_t *kpage = palloc_get_page (PAL_USER);
        if (kpage == NULL)
            return false;

        /* Load this page. */
        if (file_read (file, kpage, page_read_bytes) != (int) page_read_bytes)
        {
            palloc_free_page (kpage);
            return false;
        }
        memset (kpage + page_read_bytes, 0, page_zero_bytes);

        /* Add the page to the process's address space. */
        if (!install_page (upage, kpage, writable))
        {
            palloc_free_page (kpage);
            return false;
        }

        /* Advance. */
        read_bytes -= page_read_bytes;
        zero_bytes -= page_zero_bytes;
        upage += PGSIZE;
    }
    return true;
}

```

`install_page` 함수는 가상 메모리 공간인 `upage`를 현재 프로세스의 물리적 메모리 공간인 `kpage`에 매핑하는 역할을 한다. 먼저, `pagedir_get_page` 함수를 사용해 `upage`가 페이지 디렉토리에 존재하지 않는지 확인한다. 이후, `pagedir_set_page` 함수를 호출해 `upage`를 `kpage`에 매핑하고, 매핑 성공 여부를 반환한다. 이 함수에서 사용되는 `pagedir_get_page`와 `pagedir_set_page` 함수에 대한 상세 설명은 아래에서 다룰 예정이다.

```
static bool
install_page (void *upage, void *kpage, bool writable)
{
    struct thread *t = thread_current ();

    /* Verify that there's not already a page at that virtual
       address, then map our page there. */
    return (pagedir_get_page (t->pagedir, upage) == NULL
            && pagedir_set_page (t->pagedir, upage, kpage, writable));
}
```

이번 프로젝트의 과제 중 하나로 growing stack을 구현해야 하기 때문에, 처음 stack이 설정되는 함수에 대해 설명할 것이다. 아래의 setup_stack 함수를 보면, 먼저 위에서 설명한 palloc 함수를 이용해 빈 페이지를 할당하고 0으로 초기화한다. 그리고 install_page를 이용해 가상 주소와 물리적 주소를 매핑한다. 마지막으로, 스택 포인터인 esp를 stack의 최상단인 PHYS_BASE로 설정한다.

```
static bool
setup_stack (void **esp)
{
    uint8_t *kpage;
    bool success = false;

    kpage = palloc_get_page (PAL_USER | PAL_ZERO);
    if (kpage != NULL)
    {
        success = install_page (((uint8_t *) PHYS_BASE) - PGSIZE, kpage, true);
        if (success)
            *esp = PHYS_BASE;
        else
            palloc_free_page (kpage);
    }
    return success;
}
```

다음으로, 위에서 자주 언급된 **page directory**와 관련된 함수들에 대해 살펴보았다. 페이지 디렉토리는 이전 프로젝트 2에서 다룬 개념으로, **virtual memory space**을 **physical memory space**에 매핑하기 위해 **page table**을 관리하는 역할을 한다.

먼저, **pagedir_create** 함수는 새로운 페이지 디렉토리를 생성하는 함수이다. 이 함수는 **memcpy**를 사용해 **init_page_dir**의 **PGSIZE** 크기 데이터를 새로 생성된 **pd**에 복사한 뒤, 생성된 **pd**를 반환한다.

```
uint32_t *
pagedir_create (void)
{
    uint32_t *pd = palloc_get_page (0);
    if (pd != NULL)
        memcpy (pd, init_page_dir, PGSIZE);
}
```

```

    return pd;
}

```

`pagedir_destroy` 함수는 지정된 `page directory`에 포함된 모든 리소스를 해제(`free`)하는 역할을 한다. 이 함수는 `for` 루프를 사용해 페이지 디렉토리 내부의 각 `entry`를 순회하면서, 먼저 해당 페이지 테이블의 주소를 가져온다. 이후, 또 다른 `for` 루프를 통해 페이지 테이블의 각 엔트리를 순회하며, 각 엔트리를 `pallocc_free_page`를 사용해 해제한다. 모든 엔트리를 해제한 후, 페이지 테이블의 모든 엔트리를 순회가 완료 되면 마지막으로 해당 페이지 디렉토리 자체를 해제한다.

```

void
pagedir_destroy (uint32_t *pd)
{
    uint32_t *pde;

    if (pd == NULL)
        return;

    ASSERT (pd != init_page_dir);
    for (pde = pd; pde < pd + pd_no (PHYS_BASE); pde++)
        if (*pde & PTE_P)
        {
            uint32_t *pt = pde_get_pt (*pde);
            uint32_t *pte;

            for (pte = pt; pte < pt + PGSIZE / sizeof *pte; pte++)
                if (*pte & PTE_P)
                    pallocc_free_page (pte_get_page (*pte));
            pallocc_free_page (pt);
        }
    pallocc_free_page (pd);
}

```

`lookup_page` 함수는 인자로 전달받은 `vaddr`에 해당하는 PTE(Page Table Entry)를 찾거나 생성하는 함수이다. 이 함수에서 주의할 점은, `create` 옵션이 활성화된 경우라도 `vaddr`이 커널 영역의 주소일 경우 새 페이지 테이블을 생성할 수 없으므로 예외 처리가 필요하다는 것이다. 먼저, `vaddr`의 상위 10비트를 추출하여 이를 `pde`(Page Directory Entry) 값으로 저장한다. 만약 새로운 엔트리를 생성(`create`)해야 한다면, `pallocc_get_page`와 `pde_create` 함수를 사용하여 새로운 PDE를 생성한다. 그 후, `pde_get_pt` 함수를 통해 해당 PDE가 가리키는 페이지 테이블의 물리 메모리 주소를 가져온다. 마지막으로, `vaddr`에 해당하는 PTE를 반환한다.

```

static uint32_t *
lookup_page (uint32_t *pd, const void *vaddr, bool create)
{
    uint32_t *pt, *pde;

    ASSERT (pd != NULL);

    /* Shouldn't create new kernel virtual mappings. */

```

```

ASSERT (!create || is_user_vaddr (vaddr));

/* Check for a page table for VADDR.
   If one is missing, create one if requested. */
pde = pd + pd_no (vaddr);
if (*pde == 0)
{
    if (create)
    {
        pt = palloc_get_page (PAL_ZERO);
        if (pt == NULL)
            return NULL;

        *pde = pde_create (pt);
    }
    else
        return NULL;
}

/* Return the page table entry. */
pt = pde_get_pt (*pde);
return &pt[pt_no (vaddr)];
}

```

`pagedir_activate` 함수는 앞서 생성된 `page directory`를 활성화하는 역할을 한다.

만약 인자로 전달받은 페이지 디렉토리가 비어있거나(NULL인 경우), 해당 디렉토리가 유효하지 않다면, `init_page_dir` 함수를 호출하여 초기 페이지 디렉토리 형태로 활성화한다. 그리고 어셈블리 코드를 사용해 physical memory 주소를 virtual memory 에 mapping 해준다.

```

void
pagedir_activate (uint32_t *pd)
{
    if (pd == NULL)
        pd = init_page_dir;

    /* Store the physical address of the page directory into CR3
       aka PDBR (page directory base register). This activates our
       new page tables immediately. See [IA32-v2a] "MOV--Move
       to/from Control Registers" and [IA32-v3a] 3.7.5 "Base
       Address of the Page Directory". */
    asm volatile ("movl %0, %%cr3" : : "r" (vtop (pd)) : "memory");
}

```

다음으로, `pagedir_set_page` 함수는 매핑을 수행하는 함수로, 몇 가지 조건을 확인한 후 매핑을 진행한다. 먼저, `upage`(가상 주소)가 반드시 페이지의 시작을 가리켜야 한다. 또한, `kpage`(물리 주소) 역시 페이지의 시작 주소여야 한다. 그다음으로, `upage`는 user memory pool 에 속해야 하며, `kpage`는 물리 메모리 영역 내에 있어야 한다. 마지막으로, 페이지 디렉토리가 `init_page_dir`에 매핑되지 않도록 해야 하는데, 이는 사용자 프로세스의 페이지 디렉토리만 접근할 수 있도록 하기 위함이다.

위의 조건들을 확인한 후, 앞서 설명한 `lookup_page` 함수를 이용해 PTE를 찾고, `pte_create_user` 함수를 사용해 `kpage`를 `upage`에 매핑한다. 이때 매핑의 성공 여부는 `boolean` 값으로 반환된다.

```
bool
pagedir_set_page (uint32_t *pd, void *upage, void *kpage, bool writable)
{
    uint32_t *pte;

    ASSERT (pg_ofs (upage) == 0);
    ASSERT (pg_ofs (kpage) == 0);
    ASSERT (is_user_vaddr (upage));
    ASSERT (vtop (kpage) >> PTSHIFT < init_ram_pages);
    ASSERT (pd != init_page_dir);

    pte = lookup_page (pd, upage, true);

    if (pte != NULL)
    {
        ASSERT ((*pte & PTE_P) == 0);
        *pte = pte_create_user (kpage, writable);
        return true;
    }
    else
        return false;
}
```

`pagedir_get_page` 함수는 `uaddr`에 매핑된 physical address를 반환하는 함수이다. 먼저, `lookup_page` 함수를 호출해 `uaddr`에 해당하는 PTE(Page Table Entry)를 찾고 이를 저장한다. 그런 다음, `pte_get_page` 함수를 사용해 해당 PTE에 매핑된 physical page의 시작 주소를 가져온다. 마지막으로, `uaddr`의 오프셋(offset)을 이 physical page의 시작 주소에 더한 값을 반환하여, 최종적으로 physical page를 계산해 반환한다.

```
void *
pagedir_get_page (uint32_t *pd, const void *uaddr)
{
    uint32_t *pte;

    ASSERT (is_user_vaddr (uaddr));

    pte = lookup_page (pd, uaddr, false);
    if (pte != NULL && (*pte & PTE_P) != 0)
        return pte_get_page (*pte) + pg_ofs (uaddr);
    else
        return NULL;
}
```

`pagedir_clear_page` 함수는 `upage`와 매핑된 PTE(Page Table Entry)를 해제하는(clear) 함수이다. 이 함수는 먼저 `lookup_page` 함수를 사용해 `upage`에 해당하는 PTE를 찾는다. 이후, `invalidate_pagedir` 함수를 호출하여 페이지 매핑을 제거하고, TLB(Translation Lookaside Buffer)를 재활성화하여 최신 상태의 매핑을 유지할 수

있도록 한다. 또한, 이 함수에서도 `upage`가 페이지의 시작 주소인지와 사용자 영역의 가상 주소인지 여부를 확인하여 조건을 만족하는 경우에만 동작을 수행한다.

```
void
pagedir_clear_page (uint32_t *pd, void *upage)
{
    uint32_t *pte;

    ASSERT (pg_ofs (upage) == 0);
    ASSERT (is_user_vaddr (upage));

    pte = lookup_page (pd, upage, false);
    if (pte != NULL && (*pte & PTE_P) != 0)
    {
        *pte &= ~PTE_P;
        invalidate_pagedir (pd);
    }
}
```

먼저, 네 개의 함수를 설명하기 전에 `dirty`에 대해 알아보았다. 가상 주소가 `dirty`하다는 것은 해당 데이터가 변경되었음을 의미한다.

`pagedir_is_dirty` 함수는 `lookup_page` 함수를 사용해 `vpage`에 해당하는 PTE(Page Table Entry)를 찾은 후, 해당 PTE가 수정된 적이 있는지 `PTE_D` 비트를 확인하여 결과를 `boolean` 값으로 반환한다.

```
bool
pagedir_is_dirty (uint32_t *pd, const void *vpage)
{
    uint32_t *pte = lookup_page (pd, vpage, false);
    return pte != NULL && (*pte & PTE_D) != 0;
}
```

`pagedir_set_dirty` 함수는 위에서 언급한 `vpage`의 `PTE_D` 비트를 수정하는 함수이다. 만약 `dirty` 인수가 `true`라면 `PTE_D`를 설정하고, 그렇지 않은 경우 `PTE_D`를 해제한 뒤 `invalidate_pagedir` 함수를 호출해 TLB(Translation Lookaside Buffer)를 재활성화한다.

```
void
pagedir_set_dirty (uint32_t *pd, const void *vpage, bool dirty)
{
    uint32_t *pte = lookup_page (pd, vpage, false);
    if (pte != NULL)
    {
        if (dirty)
            *pte |= PTE_D;
        else
        {
            *pte &= ~(uint32_t) PTE_D;
            invalidate_pagedir (pd);
        }
    }
}
```

```

    }
}
}

```

`pagedir_is_accessed` 함수는 `vpage`가 접근된 적이 있는지를 확인하는 함수이다. `PTE_A` 값을 활용하여 반환하며, `PTE_A`는 CPU가 해당 `vpage`에 대해 읽기 또는 쓰기 작업을 수행했는지 나타내는 플래그이다.

```

bool
pagedir_is_accessed (uint32_t *pd, const void *vpage)
{
    uint32_t *pte = lookup_page (pd, vpage, false);
    return pte != NULL && (*pte & PTE_A) != 0;
}

void
pagedir_set_accessed (uint32_t *pd, const void *vpage, bool accessed)
{
    uint32_t *pte = lookup_page (pd, vpage, false);
    if (pte != NULL)
    {
        if (accessed)
            *pte |= PTE_A;
        else
        {
            *pte &= ~(uint32_t) PTE_A;
            invalidate_pagedir (pd);
        }
    }
}

```

지금까지 `pallocc`, `stack`, 그리고 `page directory`에 대해 알아보았다. 이제는 페이지 폴트(page fault)가 발생했을 때 어떤 동작이 수행되는지 살펴보자. 아래의 `page_fault` 함수는 `kill` 함수를 호출해 페이지 폴트가 발생한 프로세스를 즉시 종료한다. 그러나 이번 과제에서는 프로세스를 바로 종료하지 않고, 디스크에서 적절한 페이지를 메모리에 로드하는 과정을 구현할 것이다. 이에 대한 자세한 내용은 아래 `design Implementation` 파트에서 설명할 예정이다.

```

static void
page_fault (struct intr_frame *f)
{
    bool not_present; /* True: not-present page, false: writing r/o page. */
    bool write;       /* True: access was write, false: access was read. */
    bool user;        /* True: access by user, false: access by kernel. */
    void *fault_addr; /* Fault address. */

    asm ("movl %%cr2, %0" : "=r" (fault_addr));

    intr_enable ();

    /* Count page faults. */

```

```

    page_fault_cnt++;

    /* Determine cause. */
    not_present = (f->error_code & PF_P) == 0;
    write = (f->error_code & PF_W) != 0;
    user = (f->error_code & PF_U) != 0;

    printf (...);
    kill (f);
}

```

Design Implementation

1. Frame table

Basics

위에서 설명한 바와 같이 Frame은 물리 메모리의 연속적인 영역을 나타낸다. Frame table은 이러한 물리 메모리의 frame들을 효율적으로 관리하기 위한 데이터 구조이다. Frame table의 각 entry는 특정 frame에 로드된 페이지의 포인터를 포함하며, 각 frame이 어떤 페이지와 매핑되어 있는지를 명시한다. 기존 PintOS에서는 page(즉, 가상 메모리)에 대한 관리만을 위해 page table이 구현되어 있을 뿐, frame table에 대한 구현은 존재하지 않는다.

Limitations and Necessity

Frame table이 없으면 물리 메모리를 효율적으로 관리하기 어렵다. 기존 PintOS에서는 특정 프로세스가 어떤 frame을 사용하는지 추적할 방법이 없으며, 메모리 부족 상황에서 swap을 수행할 수 없는 한계가 존재한다. Frame table을 도입하면 사용 중인 frame을 추적할 수 있으며, 비어 있는 frame이 없는 경우 eviction 정책을 통해 이미 할당된 페이지를 swap out하여 여유 공간을 확보할 수 있다. 이를 통해 새 페이지를 할당하고 프로세스의 정상적인 동작을 유지할 수 있다. (swap 에 관한 내용은 6번에서 자세히 설명하였다)

Blueprint (Proposal)

Data Structure

Frame table은 list를 사용해 구현할 예정이다. 이 리스트는 각 frame을 나타내는 frame table entry로 구성된다. frame table entry는 새로운 구조체로 선언할 것이며, 해당 구조체는 physical memory frame 주소와 virtual page 주소를 분리하여 저장한다. 또한, list_elem 필드를 통해 frame table 리스트에 연결 리스트 형태로 저장할 예정이다. Frame table은 global 데이터이므로, 이를 보호하기 위해 lock을 사용하여 atomic하게 접근하도록 할 예정이다.

Data Structure Definition

```

// Frame table
struct list FrameTable;

// Frame table entry

```

```

struct FrameTableEntry {
    struct list_elem FTE_elem; // List element for connection
    void *frame_adr;           // Address of the physical frame
    void *page_adr;            // Address of the mapped virtual page
};

// Lock for frame table
struct lock ftLock;

```

Pseudo Code or Algorithm

Eviction Policy

Frame table 관리에는 Clock Algorithm을 기반으로 한 eviction policy를 사용하려고 한다. 특정 엔트리가 비어 있는 경우에는 새로운 페이지 포인터를 해당 엔트리에 추가한다. 하지만 모든 엔트리가 이미 특정 페이지 포인터를 보유한 경우 eviction policy에 따라 지정된 페이지를 swap out한 후 새로운 페이지를 할당한다. 이를 구현하기 위해 다음과 같은 세 가지 함수를 추가 할 예정이다:

- **init_ft**: Frame table과 관련된 전역 데이터 구조(FrameTable과 ftLock)를 초기화한다.
- **alloc_new_frame**: 요청된 **page_adr**와 매핑되는 새 유효 **frame_adr**을 할당한다. 성공 시 FrameTableEntry에 해당 정보를 저장하고 요청된 페이지 주소에 매핑된 frame 주소를 반환한다.
- **free_frame**: **alloc_new_frame**을 통해 할당된 frame과 매핑된 페이지를 할당 해제하고 관련 엔트리를 정리한다.

2. Lazy loading

Basics

Lazy loading은 access 요청이 올 때마다 필요한 page data를 memory에 저장하는 것을 말한다. 기존의 PintOS 는 page의 모든 executable이 memory에 load되는 방식을 사용한다. 아래의 load_segment 를 보면 while loop 를 통해서 해당 세그먼트의 모든 데이터가 페이지 단위로 메모리에 load 되는 것을 볼 수 있다. 그리고 install_page 를 통해 user space 의 virtual address upage 와 kernel 에서 할당한 physical page kpage 를 매핑해 준다.

```

static bool
load_segment (struct file *file, off_t ofs, uint8_t *upage,
              uint32_t read_bytes, uint32_t zero_bytes, bool writable)
{
    ...

    file_seek (file, ofs);
    while (read_bytes > 0 || zero_bytes > 0)
    {
        ...
        /* Load this page. */
        if (file_read (file, kpage, page_read_bytes) != (int) page_read_bytes)
        {
            palloc_free_page (kpage);
            return false;
        }
    }
}

```

```

    }
    memset (kpage + page_read_bytes, 0, page_zero_bytes);

    /* Add the page to the process's address space. */
    if (!install_page (upage, kpage, writable))
    {
        palloc_free_page (kpage);
        return false;
    }
    ...
}
return true;
}

```

Limitation and Necessity

모든 executable을 한번에 load하면 불필요한 data까지 memory에 load되어 메모리 공간 낭비가 심각하다는 문제가 있다. 또한 page fault를 처리하는 방법이 process termination과 kernel panic에 의존하기 때문에 효율적이지 못하다. 반면 Lazy loading은 필요한 시점에 필요한 데이터만 메모리에 로드하기 때문에 메모리를 효율적으로 사용할 수 있다는 장점을 갖는다.

Blueprint (Proposal)

Data Structure

Lazy loading이 가능하다는 것은 언제 어떤 page가 필요한지 관리할 수 있음을 의미하며, 이는 각 page의 정보를 추적할 수 있는 데이터 구조가 필요함을 뜻한다. 각 page의 정보를 저장하는 구조체는 아래와 같다.

```

struct page_table {
    struct file *f;           // 해당 page가 속한 파일
    off_t pte_fo;            // 파일 offset
    bool isWritable;         // 쓰기 가능 여부
};

```

정리하자면, **page_table** 구조체는 해당 page가 어느 file의 일부인지, file 내의 offset, 그리고 쓰기 가능 여부와 같은 정보를 저장한다.

Pseudo Code or Algorithm

Lazy loading은 page fault 발생 시 시작된다. page fault가 발생했을 때, fault가 난 가상 메모리 주소가 물리 메모리에 이미 load되어 있다면 그대로 반환한다. 그렇지 않은 경우 해당 페이지를 physical memory에 load한다. 이 과정이 lazy loading이며, 일종의 fault handler이다.

페이지 데이터를 가져오는 출처는 page의 type에 따라 다르다. 예를들어 일부 경우는 swap 영역에서 데이터를 가져오고, 그 외의 경우 disk에서 file을 load한다. swap 영역에서의 동작에 대해서는 6번에서 자세히 설명하였다. 이 과정은 disk에서 데이터를 읽어오는 함수와 swap하는 함수를 호출하여 physical memory로의 load를 수행한다.

- `load_page_from_disk`: disk에서 file 데이터를 load하는 함수
- `swap_page`: swap 영역에서 데이터를 가져오는 함수 (6번에서 추가 설명)

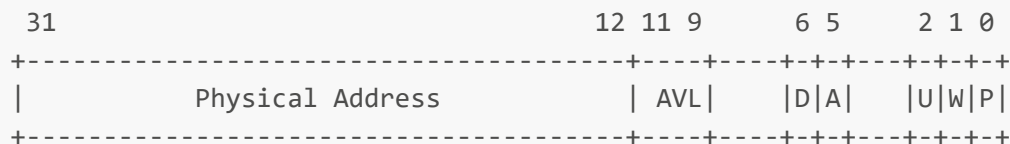
3. Supplemental page table

Basics

Supplemental page table은 기존 page table을 보완하는 데이터 구조로서 virtual memory의 정보를 관리할 수 있도록 한다. 따라서 virtual memory의 VA(virtual address)에 해당하는 정보를 하나의 구조체로 묶어 entry `spte`를 구성하고 이러한 entry를 `spt`(supplemental page table)에서 관리할 수 있도록 한다.

Limitation and Necessity

original pintos의 `pte` format은 다음과 같다.



현재 구현된 Page Table은 `availability`(AVL), `present`(P), `read/write`(W), `user/supervisor`(U), `accessed`(A), `dirty`(D)의 정보를 0~11bit에 저장한다. 그러나 lazy loading과 같은 기능을 지원하고 page fault를 처리하기 위해서는 추가적인 정보를 포함할 필요가 있다. 현재 pintos는 virtual memory를 관리할 수 있는 table이 함수가 없으므로 지정된 virtual memory를 `spt`를 통해 관리할 수 있도록 한다.

Blueprint (proposal)

Data structure

`spt`의 entry `spte`는 특정 virtual memory의 정보를 갖기 때문에 정해진 VA에 대한 다음과 같은 정보를 가져야 한다. 따라서 아래 요소들은 `spte` 구조체의 필드로 존재한다.

- `va_vpn`: Virtual Page Number
- `va_offset`: Offset
- `isLoad`: physical memory에 load시 1, otherwise 0
- `file`: mapping된 file의 pointer
- `srcType`: file의 source가 binary인지, mapping file인지, swap disk인지 구분한다.
- `isWritable`: 해당 VA이 writable하면 1, otherwise 0

그러나 제공된 VA는 매우 많기 때문에 이러한 개별 VA에 대응되는 `spte`를 list로 관리하면 비효율적이다. 따라서 우리는 pintos에서 이미 구현되어 제공되는 `hash table`을 이용해 `spt`를 hash table로 구현하기로 한다. 따라서 `struct spte`는 아래의 필드도 포함하여야 한다.

- `spte_hashElem`: `spte`의 hash table element

pseudo code or algorithm

`spt`는 아래와 같이 thread의 필드로 선언한다.

```
struct thread{
/* thread.h의 thread 구조체 선언부 */
    struct hash spt;
    ...
};
```

hash algorithm을 이용하나, pintos에 미리 구현된 hash table을 사용하며 아래의 함수들로 spt를 관리한다.

- `get_spte_key`: spte의 hash key를 반환한다.
- `get_spte`: spte의 evict등을 위해 spt에서 spte를 return한다.
- `init_spt`: `hash_init`을 이용해 spt를 초기화한다.
- `get_new_spte`: malloc을 이용해 새로운 frame을 할당 후 spt에 `has_insert`로 삽입하도록 한다.

4. Stack growth

Basics

Growing stack은 프로세스의 요청에 따라 유한하게 크기를 동적으로 늘릴 수 있는 메커니즘이다. 그러나 기존 Pintos는 4KB 크기의 fixed-size stack만을 지원하며, dynamic하게 크기를 조정할 수 없다. 이를 해결하기 위해, 프로세스 요청에 따라 4KB보다 더 큰 stack을 제공할 수 있도록 수정해야 한다.

Limitation and Necessity

기존 Pintos는 고정된 4KB의 fixed-stack size를 제공하며, 이를 초과한 stack 요청이 있을 경우 page fault가 발생하도록 설계되어 있다. 그러나 fixed-size stack은 local variable과 argument 수에 제한을 두게 되며, 이는 실제 프로그램 실행에서 유용하지 않다. 따라서 dynamic하게 stack size를 확장할 수 있는 기능이 필요하다.

Blueprint (Proposal)

Data Structure

Stack grow 기능을 구현하기 위해 각 thread의 stack pointer(esp) 값을 추적해야 하므로, 기존 thread 구조체에 esp 포인터를 추가한다. 별도의 새로운 구조체를 선언하지는 않는다.

Pseudo Code or Algorithm

Page fault 발생 시 stack grow가 필요한 상황을 확인하고, 조건에 맞으면 stack을 확장하도록 구현한다. 이를 위해 다음과 같은 함수를 구현하거나 수정한다:

- `is_stack_access(addr, esp)`: Page fault가 발생한 주소(addr)가 현재 thread의 esp와 충분히 가까운지 확인한다. 조건을 만족할 경우 stack grow를 허용한다.
- `grow_stack(fault_addr)`: Page fault가 발생한 주소를 기준으로 stack 영역을 확장한다. 이때, free page를 supplemental page table을 통해 할당받아 물리 메모리에 매핑한다.
- `page_fault`(기존 함수 수정): Page fault가 발생했을 때 fault address를 확인하고, stack grow 조건을 만족하면 `grow_stack`을 호출하여 stack을 확장한다. 조건에 부합하지 않을 경우 기존 page fault 처리 방식을 유지한다.

5. File memory mapping

Basics

File memory mapping은 파일 내용을 page(가상 메모리)에 매핑하여 프로세스가 메모리에서 파일 데이터에 직접 접근할 수 있도록 하는 메커니즘이다. 기존 PintOS는 파일 입출력을 `file_read` 및 `file_write`를 통해서만 처리하도록 구현되어 있다. 이번 프로젝트에서는 페이지를 파일에 매핑하는 file memory mapping을 구현할 것이다.

Limitation and Necessity

기존 PintOS는 `file_read`와 `file_write`를 통해서만 파일에 접근할 수 있기 때문에 효율성이 떨어지며, shared resource에 대한 지원이 부족하다. 이를 개선하기 위해 `mmap` syscall과 `munmap` syscall을 구현하여 파일 데이터를 디스크에서 메모리로 로드하고, 이를 프로세스가 직접 메모리에서 접근할 수 있도록 한다. 이를 통해 효율적인 메모리 사용이 가능하도록 할 예정이다.

Blueprint (Proposal)

Data Structure

File memory mapping(이하 FMM)은 여러 개의 매핑을 동시에 관리해야 하므로, 현재 관리 중인 모든 FMM을 `FMMList`라는 리스트에 저장한다. 이 리스트는 각 개별 FMM을 연결하는 `list_elem`을 포함하며, 각 FMM은 고유한 데이터 구조로 선언된다. 이를 위해 `FMM` 구조체를 정의하고, 다음과 같은 필드를 포함한다.

```
struct FMM {
    int fmm_id;           // 고유 ID
    struct file *f;       // 매핑된 파일 포인터
    struct list_elem FMM_elem; // FMMList와 연결하는 필드
};
```

이러한 FMM 정보는 프로세스별로 관리되므로, `thread` 구조체에 FMM 리스트를 추가하여 해당 프로세스의 모든 매핑 정보를 추적할 수 있도록 한다.

Pseudo Code or Algorithm

File memory mapping 관리 알고리즘은 다음의 syscall 함수들을 통해 구현된다:

- `sys_mmap`: 특정 파일을 메모리에 매핑한다. 요청된 파일에 대해 새로운 FMM 구조체를 생성하고, 해당 구조체를 `FMMList`에 추가한다. 이를 통해 특정 프로세스가 접근하는 모든 FMM을 효율적으로 관리할 수 있다.
- `sys_munmap`: 매핑된 파일의 메모리 매핑을 해제한다. `sys_mmap`에 의해 생성된 FMM을 해제하기 위해 `FMMList`를 순회하면서 매핑된 FMM 구조체를 찾고, 이를 할당 해제한다.

6. Swap table

Basics

Swap에는 두 가지 주요 동작이 있다. Swap-in은 page fault가 발생했을 때 disk의 특정 영역에서 페이지를 가져와 memory의 frame에 로드하는 과정을 말하며, Swap-out은 swap-in을 수행하기 위해 정해진 policy에 따라

frame에 로드되어 있던 페이지를 evict하여 swap disk에 저장하고 free frame을 확보하는 과정을 말한다. Swap disk를 관리하기 위해 사용하는 데이터 구조를 swap_table이라 한다.

Swap 기능을 통해 virtual address를 사용한 address translation은 프로세스 관점에서 무한히 넓은 메모리 공간을 제공하는 것처럼 보이며, 실제 physical memory보다 넓은 영역을 사용할 수 있도록 한다.

Limitation and Necessity

기존 PintOS는 swap 기능이 구현되어 있지 않다. 이로 인해 메모리가 부족한 경우 프로세스가 종료되도록 설계되어 있다. 이는 메모리 활용의 유연성을 크게 제한한다.

Blueprint (Proposal)

Data Structure

Swap 영역의 사용 여부를 추적하기 위해 bitmap을 사용하며, 이를 swap_table로 정의한다. 특정 bit가 1로 설정된 경우 해당 영역이 swap-out 가능하다는 것을 의미하도록 한다. 이 외에도 disk와 swap 작업의 동기화를 관리하기 위해 다음과 같은 구조체를 사용한다.

아래의 내용 맞는지 확인 필요

- `swap_disk`: swap 영역이 위치한 디스크를 관리.
- `swap_lock`: swap 작업이 동기화되도록 보호.

```
struct swap_disk {
    struct block *disk;      // Disk block 포인터
    size_t size;             // Swap 영역 크기
    struct bitmap *swap_map; // Bitmap으로 swap 상태 관리
};

struct lock swap_lock;      // Swap 작업 동기화
```

Pseudo Code or Algorithm

Swap-in과 Swap-out 각각의 동작을 다음과 같이 정의한다:

- `swap_out(frame)`:
 - Disk에서 사용할 빈 슬롯을 bitmap에서 찾는다.
 - 해당 페이지를 디스크에 저장하고, Supplemental page table을 업데이트한다.
 - Physical memory에서 페이지 매핑을 제거한다.
- `swap_in(page)`:
 - Physical memory의 frame을 새로 할당한다.
 - Disk에서 swap table을 참조하여 해당 데이터를 읽어온다.
 - 데이터를 메모리에 로드한 후, Supplemental page table을 업데이트한다.
- `evict_page()`:

- Clock algorithm을 통해 swap-out할 페이지를 선택한다.
- 선택된 페이지를 `swap_out` 함수로 처리하여 공간을 확보한다.

7. On process termination

Basics

프로세스가 종료(exit)할 때, 할당된 모든 자원을 해제하여 메모리 누수를 방지해야 한다. 여기서 자원이란 위에서 구현한 frame, supplemental page table 등을 포함한다. 또한, 위의 기능들을 구현하면서 사용된 lock도 반드시 release해야 한다. Lock을 release하지 않고 프로세스를 종료하면 이후 deadlock이 발생할 수 있다.

Limitation and Necessity

위의 1~6번에서 설명한 것처럼, 몇몇 구조체와 데이터는 기존 PintOS에 구현되어 있지 않다. 따라서 이러한 리소스를 새롭게 선언하거나 기존 thread 구조체의 필드로 추가하였다. 그러나 기존 PintOS는 이러한 추가된 리소스에 대한 해제를 구현하고 있지 않다. 이를 해결하기 위해 `process_exit` 함수를 수정하여, 종료 시점에서 모든 리소스를 적절히 release하도록 구현할 것이다.

Blueprint (Proposal)

Data Structure

`all_LockList`: 모든 lock을 관리하는 리스트 데이터 구조로, 생성된 lock을 추적하기 위해 global하게 선언한다. 이를 통해 acquire되거나 release된 모든 lock을 추적할 수 있다. 그리고, Lock 구조체의 필드에 `list_elem`을 추가하여 `all_LockList`에 연결할 수 있도록 한다. 그 외의 추가적인 구조체나 변수는 필요하지 않다.

Pseudo Code or Algorithm

- `release_frame_resources` : 현재 thread가 사용한 frame을 모두 해제한다. Frame table에서 해당 thread의 frame들을 제거한다.
- `release_supplemental_page_table` : Thread에 연결된 supplemental page table 엔트리를 순회하며 모든 가상 메모리 매핑 정보를 삭제한다.
- `release_file_mappings` : Memory-mapped 파일 목록을 순회하며 모든 매핑을 해제하고 관련 자원을 반환한다.
- `release_all_locks**` : `all_LockList`를 순회하며 해당 thread가 보유한 모든 lock을 release한다.